

WIKIHOW PERKS

Summary

Overview4

Interpreter5

Definition5

Game Data5

Any Module Data5

BufferModule Data5

GaugeModule Data6

Event Specific Data6

Attack Data6

TargetUnit Data6

Owner Data7

DynamicStatModifier Data7

Securities7

Skill Data7

Interpreted Methods7

TokenVariables9

Definition9

XSD9

Example9

LocArguments10

Definition10

XSD10

Types11

LocArgument11

LocalizedArgument11

StatArgument11

RestoreStatArgument12

StatusArgument12

AttackTypeArgument13

Example13

View14

Definition14

XSD14

Child elements14

SkillsToShow14

CompendiumEntries15

DisplayInHUD16

DisplayBonusBeforePurchase17

FeedbackActivationConditions17

Modules18

Definition18

Types of Modules18

BufferModule18

GaugeModule18

Conditions19

Definition19

Types19

Example19

Actions20

Definition20

Types20

Events23

Definition23

Types23

Example24

Effects25

Definition25

Types25

Miscellaneous30

AttackSkillEffects30

ModifiableStat31

RegenerableStat32

SkillComputationStat32

PerkTargeting33

Status34

SlotId34

Examples: Creation of a perk step by step35

Introduction35

Level: very easy – GlassCannon36

First Step: Id and variables36

Second Step: Modules36

Third Step: Localization36

Level: Fairly easy – NightOwl38

First Step: Id and variables38

Second Step: Modules38

Third Step: View41

Fourth Step: Localization42

Level: Medium – Duelist44

First Step: Id and variables44

Second Step: Modules44

Third Step: View50

Fourth Step: Localization51

Conclusion53

Version**Erreur ! Signet non défini.**

Overview

The perk system is an event-based system, every perk reacts to a set of events that were implemented for our initial needs.

Each event carry data, that is fulfilled with the expected information beforehand. Different event might not carry the same data (or even data at all), as it would not make sense. For example, the OnMovementEnd event does not carry the same data as the OnTargetHit event.

By design, when dealing with perks that are updated upon kill/hit, we usually want to update the perk's values between two hits. This allow a perk to be consistent within the same aoe or propagation, and allows us to have clear visual feedback timing without spamming too much the player.

Interpreter

Definition

The Last Spell has a home-made parser and interpreter that can be used to access some runtime information, as long as the data is correctly fulfilled.

In that regard, perks are “interpretable objects”. It means that you have access in XML to some information that are stored in your current perk. Perks always contains some data, such as its owner or global game information (like the CurrentNightHour or the DayNumber). In addition to this information, depending on the perk definition, you may also have access to BufferModule information and/or information linked to an event (like attack data if you have an OnHit type of event).

One thing the interpreter can also do: MATH. It handles the parenthesis, addition, subtraction, product, division, modulo. It can also handle bools in certain cases: equal (=), different (!), superior to (>).

This interpreter is **not** the best in the world, and there are quite a few things it can’t handle. Nonetheless, we tried to give access to as much data as we could, and that we needed for our own perks design. You should have more than enough things to play around for your own perk designs.

Here is some of the information available to you, trying to describe an exhaustive list:

Game Data

- bool IsNightCycle
- bool IsDayCycle
- bool IsEnemyTurn
- bool IsPlayableTurn
- int CurrentNightHour
- int DayNumber

Any Module Data

- bool IsActive
- CastSkillEffect -> this is used in pair with the CastSkill effect. It will allow you to access data about the skill, for example its range. This is used only for HumanBallista so far, in order to display the perk skill range.

BufferModule Data

Only the first Module is accessible so far. To do so, you have to Specify “Module.” before the desired information, like so: “Module.Buffer1”

- *Default Module data (see above)*
- int Buffer
- int Buffer2
- int Buffer3

GaugeModule Data

Only the first Module is accessible so far. To access its data, you have to specify "Module." before the desired information, like so: "Module.GaugeValue"

- Everything contained BufferModule (see above)
- int GaugeValue
- Int GaugeMaxValue
- Int ValueOffset

Event Specific Data

You do not need any prefix to access this data, however it is highly recommended for security reason to check if HasPerkData (see Securities) is true before using any of this.

- Securities (bool): HasSkill, HasTargetTile, IsTargetUnit, IsTargetEnemyUnit, IsTargetPlayableUnit, IsTargetBuilding, IsCasterUnit, IsCasterEnemyUnit, IsCasterPlayableUnit
- Bool OwnerIsCaster, OwnerIsTarget
- Int DistanceFromOwnerToTarget, DistanceFromCaster (distance between caster and targeted tile, **not** the target hit), TargetUnitDistanceFromCaster
- Statuses (bool): AppliedStun, AppliedPoison, AppliedCharge, AppliedBuff, AppliedDebuff, AppliedContagion, AppliedNegativeAlteration, TargetWasStunned, TargetWasPoisoned, TargetWasCharged, TargetWasBuffed, TargetWasDebuffed, TargetWasContagious, TargetHadNegativeAlteration
- Bool IsTriggeredByPerk

Attack Data

Attack Data is contained in the Event Specific data container, therefore you should make sure that this data exists before using it (see Event Specific Data). To access Attack data, you have to specify "AttackData." before the desired information, like so: "AttackData.IsCrit"

- Float ArmorDamage, HealthDamage, TargetRemainingArmor, TargetRemainingHealth, TotalDamage, HealthDamageIncludingOverkill, OverkillDamage
- Bool Dodged, IsCrit, IsInstantKill, IsPoison, IsPropagatedTile

TargetUnit Data

TargetUnit is contained in the Event Specific data container, therefore you should make sure that this data exists before using it (see Event Specific Data). To access TargetUnit data, you have to specify "TargetUnit." before the desired information, like so: "TargetUnit.Health"

- Float Stats: Accuracy, Armor, ArmorTotal, ExperienceGain, HealingReceived, Health, HealthTotal, IsolatedAttacks, Mana, ManaTotal, MovePoints, OpportunisticAttacks, Reliability, Critical, Block
- Bool Statuses: IsStunned, IsPoisoned, IsCharged, IsBuffed, IsDebuffed, IsContagious, IsImmune, HasNegativeAlteration
- int DistinctNegativeAlterationsCount
- bool IsIsolated
- bool IsAdjacentToBuilding
- int InjuryStage
- E_AttackType LastSkillType

- Bool WillDieByPoison

Owner Data

Owner is always accessible since it is contained into the perk itself. To access Owner data, you have to specify "Owner." Before the desired information, like so: "Owner.IsBackProtectionValid"

- Everything contained in TargetUnit (see TargetUnit Data)
- Stats: ActionPoints, HealthRegen, MagicalDamage, PhysicalDamage, RangedDamage, MagicDamageReductionModifier, UnlockedArmorTotal, UnlockedManaTotal
- HeroesCount
- TrinketLevels
- ClosestAllyDistance
- IsBackProtectionValid (BackProtection only works with specific buildings, that's why we don't use IsAdjacentToBuilding)
- IsManaCostLocked (example: BloodMagic perk usage)
- Int TileCrossedThisTurn
- Int TotalMomentumTilesCrossedThisTurn
- Int MomentumTilesActive

DynamicStatModifier Data

This is a very specific data container, only used in our perk Versatility.

- Stat HighestDamageType, FirstOtherDamageType, SecondOtherDamageType
- Float HighestDamageValue, FirstOtherDamageValue, SecondOtherDamageValue

Securities

- HasPerkData
- HasDynamicStatModifier

Skill Data

- Bool ProvidedByItem
- Bool IsSingleTarget
- Bool IsMagicalDamage, IsPhysicalDamage, IsRangedDamage, DamageTypeDiffersFromOwnerLastSkill
- Int ManaCost, ActionPointsCost, HealthCost, MovePointsCost (those are the **actual** costs)
- Int BaseManaCost, BaseActionPointsCost, BaseHealthCost, BaseMovePointsCost (those are the initial costs of the skill, without looking at any perk or modifiers).
- Bool IsAttack, IsAttackOrExecute
- Float MinRange, BaseMaxRange, MaxRange
- Bool HasMomentum, HasBuff, HasContagion, HasDebuff, HasPoison, HasStun, HasNegativeAlteration, HasPropagation, HasExecute, HasVision
- Bool IsManaCostLocked, IsHealthCostLocked (see *ComputationStatLocker* in the *Effects* section)

Interpreted Methods

You also have access to some method, useful for basic operations.

Here is an exhaustive list of the available methods:

- `Clamp(a, b, c)`
- `Exp(a)`
- `Floor(a)`
- `Ceil(a)`
- `Round(a)`
- `Log(a)`
- `Max(a, b)`
- `Min(a, b)`
- `Pow(a, b)`
- `Sqrt(a)`
- `BooleanChoice(a, b, c)` -> this one is the only “home-made” one. It basically represents a ternary operation, meaning if `a` is true, then it returns `b`, else it returns `c`. It is very useful for dynamic values, such as perks that reacts differently regarding the damage type used (Duelist perk).
- `Random(a, b)`
- `RandomInt(a, b)`

TokenVariables

Definition

Token Variables are shortcuts that can be used in most of the values you'll see later. These are essentially just Dictionaries entries, and our parser go through them once, from top to bottom. This means order is important. Also, it is a simple `string.replace()` call, therefore you should be very careful with the naming of the keys. I recommend using very explicit and unique key names.

TokenVariables are optional.

XSD

```
<xs:element name="TokenVariables" minOccurs="0" maxOccurs="1">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="TokenVariable" type="KeyValuePair" minOccurs="0" maxOccurs="unbounded"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>
```

Example

```
<PerkDefinition Id="BigGameHunter">
  <TokenVariables>
    <TokenVariable Key="HealthTotalConditionIsValid" Value="TargetUnit.HealthTotal > HealthTotalThreshold - 1"/>
    <TokenVariable Key="TotalFlatDamage" Value="TargetUnit.HealthTotal * (HealthPercentageDamage / 100)"/>
    <TokenVariable Key="HealthPercentageDamage" Value="15"/>
    <TokenVariable Key="HealthTotalThreshold" Value="300"/>
  </TokenVariables>
```

As you can see, `HealthPercentageDamage` is defined **after** `TotalFlatDamage`. It is important to do so, as our parser goes from top to bottom, meaning it will replace any occurrence of `TotalFlatDamage` by its value before replacing the occurrences of `HealthPercentageDamage`.

```
<PerkDefinition Id="ManaGrowth">
  <TokenVariables>
    <TokenVariable Key="TotalManaBonus" Value="Owner.Level * ManaMultiplier"/>
    <TokenVariable Key="ManaMultiplier" Value="2"/>
  </TokenVariables>
```

LocArguments

Definition

LocArguments are used for the description of the perk. Our Localization are written using arguments written as such: {0}, {1}, {2}... The LocArguments specified here are therefore replacing those arguments. Once again, their order is important, since the first element in LocArguments is automatically attributed the index 0, whereas the second element has the index 1 and so on...

LocArguments are both available in the Localized text for the description, as well as in the localized text for the detailed portion of the description (example: Berserk has two parts in its description: one that describes the perk, and another one that display its current bonus. The latest only show after purchase). By convention, detailed arguments are located at the bottom. In the localization file, the detailed description corresponds to the "PerkEffectInformations_PerkId" key.

LocArguments are optional.

XSD

```
<xs:element name="LocArguments" minOccurs="0" maxOccurs="1">
  <xs:complexType>
    <xs:sequence>
      <xs:choice minOccurs="1" maxOccurs="unbounded">
        <xs:element name="LocArgument" type="LocArgument" minOccurs="0"/>
        <xs:element name="LocalizedArgument" type="LocArgument" minOccurs="0"/>
        <xs:element name="StatArgument" type="StatArgument" minOccurs="0"/>
        <xs:element name="RestoreStatArgument" type="RestoreStatArgument" minOccurs="0"/>
        <xs:element name="StatusArgument" type="StatusArgument" minOccurs="0"/>
        <xs:element name="AttackTypeArgument" type="AttackTypeArgument" minOccurs="0"/>
      </xs:choice>
    </xs:sequence>
  </xs:complexType>
</xs:element>
```

Types

LocArgument

LocArgument are the simplest LocArgument possible. It has a mandatory value, everything else is optional. Value is not edited in any way before being used by the Localization text format.

Interpreted attribute is used to indicate that the value should be parsed and interpreted, meaning it contains value from runtime data.

Style attribute is used to apply a predefined style to the value (is applied to the entire string, meaning prefix and suffix will be included inside the Style tag).

Prefix attribute is used to add something before value (can be useful to add a “+” without interfering with the parsing/interpretation of the attribute Value).

Suffix attribute is used to add something after value (can be useful to add a “%” without interfering with the parsing/interpretation of the attribute Value).

```
<!-- LOCARGUMENTS -->
<xs:complexType name="LocArgument">
  <xs:attribute name="Value" type="xs:token" use="required"/>
  <xs:attribute name="Interpreted" type="xs:boolean" use="optional"/>
  <xs:attribute name="Style" type="xs:token" use="optional"/>
  <xs:attribute name="Prefix" type="xs:token" use="optional"/>
  <xs:attribute name="Suffix" type="xs:token" use="optional"/>
</xs:complexType>
```

LocArgument should be used to print counters, or anything that has no link to a stat, a skill, a skill effect....

LocalizedArgument

LocalizedArgument is the exact same scheme as LocArgument with a single difference being before being used by the Localization text format, we try to Localize the Value (including prefix and suffix).

Example:

```
<LocalizedArgument Prefix="SkillName_" Value="SkillId" Style="Skill"/>
```

This will return the Localized name for the specified skill id before being sent to the perk description.

StatArgument

StatArgument is meant to be used with Stats, as its name suggests. Everything is optional, but it should have at least a stat, either in its Stat attribute or in its InterpretedStat attribute.

StatArgument will automatically give you a stylized result, containing the value as well as the icon and name of the stat. If you do not want the value to be stylized according to its stat (example: fatality perk does not stylize its health value)

```
<xs:complexType name="StatArgument">
  <xs:attribute name="Value" type="xs:token" use="optional"/>
  <xs:attribute name="Interpreted" type="xs:boolean" use="optional"/>
  <xs:attribute name="Style" type="xs:token" use="optional"/>
  <xs:attribute name="Stat" type="NativeRandomizedStat" use="optional"/>
  <xs:attribute name="InterpretedStat" type="xs:token" use="optional"/>
  <xs:attribute name="DisplaySign" type="xs:boolean" use="optional"/>
</xs:complexType>
```

RestoreStatArgument

RestoreStatArgument is almost the same as StatArgument. The only difference being it will automatically add the icon and localized text “Restore” before the value and stat name (if DisplayRestoreText is equal to true).

Also, if the attribute ModifiedValue is true, it will try to print the final value instead of the base value: the only relevant case is for Health, as healing an amount is modified by the HealingReceived stat. Therefore, if ModifiedValue is true, it will take the HealingReceived stat from the perk owner into account when printing the value.

```
<xs:complexType name="RestoreStatArgument">
  <xs:complexContent>
    <xs:extension base="StatArgument">
      <xs:attribute name="DisplayRestoreText" type="xs:boolean" use="optional"/>
      <xs:attribute name="ModifiedValue" type="xs:boolean" use="optional"/>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>
```

StatusArgument

StatusArgument is meant to be used with Status, as its name suggests. The only thing mandatory is the Status attribute.

This will automatically format the argument according to the stat.

Value is only used for Buff, Debuff and Poison. Stat is used for Buff and Debuff. TurnsCount is used for every status. Chance is only relevant to Stun. ModifiedValue applies the StunChanceModifier stat to the value when dealing with Stun and applies the PoisonDamageModifier stat to the value when dealing with Poison.

```
<xs:complexType name="StatusArgument">
  <xs:attribute name="Value" type="xs:token" use="optional"/>
  <xs:attribute name="Interpreted" type="xs:boolean" use="optional"/>
  <xs:attribute name="Style" type="xs:token" use="optional"/>
  <xs:attribute name="Status" type="StatusExtended" use="required"/>
  <xs:attribute name="Stat" type="NativeRandomizedStat" use="optional"/>
  <xs:attribute name="TurnsCount" type="xs:token" use="optional"/>
  <xs:attribute name="Chance" type="xs:token" use="optional"/>
  <xs:attribute name="ModifiedValue" type="xs:boolean" use="optional"/>
</xs:complexType>
```

AttackTypeArgument

AttackTypeArgument is meant to be used with AttackTypes, as its name suggests. Everything is optional, but it should have at least n AttackType, either in its AttackType attribute or in its InterpretedAttackType attribute.

The returned value will automatically be formatted with the style linked to the AttackType, but you can override this by using another style in the Style attribute. The value always contains the Localized name of the AttackType.

```
<xs:complexType name="AttackTypeArgument">
  <xs:attribute name="Style" type="xs:token" use="optional"/>
  <xs:attribute name="InterpretedAttackType" type="xs:token" use="optional"/>
  <xs:attribute name="AttackType" use="optional">
    <xs:simpleType>
      <xs:restriction base="xs:string">
        <xs:enumeration value="None"/>
        <xs:enumeration value="Ranged"/>
        <xs:enumeration value="Physical"/>
        <xs:enumeration value="Magical"/>
        <xs:enumeration value="Adaptative"/>
      </xs:restriction>
    </xs:simpleType>
  </xs:attribute>
</xs:complexType>
```

Example

```
<PerkDefinition Id="Berserk">
  <TokenVariables>
    <TokenVariable Key="DamageBonusThisTurn" Value="Module.Buffer2 * DamageBonusByHit"/>
    <TokenVariable Key="CurrentDamageBonus" Value="(Module.Buffer + Module.Buffer2) *
DamageBonusByHit"/>
    <TokenVariable Key="DamageBonusByHit" Value="3"/>
  </TokenVariables>
  <LocArguments>
    <StatArgument Stat="OverallDamage" Value="DamageBonusByHit"/>
    <StatArgument Stat="OverallDamage" Value="CurrentDamageBonus" Interpreted="true"/>
  </LocArguments>
```

View

Definition

The view element is used for the display of the perk in the UI. This regroups the perk tree, the display at the bottom of the screen as well as the display alongside the attack estimation, on the right side of the screen. It is used to define the behavior on hover, as well as various counters or highlights conditions.

XSD

```
<xs:element name="View" minOccurs="0" maxOccurs="1">
  <xs:complexType>
    <xs:all>
      <xs:element name="SkillsToShow" type="SkillsToShow" minOccurs="0" maxOccurs="1"/>
      <xs:element name="CompendiumEntries" type="CompendiumEntries" minOccurs="0" maxOccurs="1"/>
      <xs:element name="DisplayInHUD" type="DisplayInHUD" minOccurs="0" maxOccurs="1"/>
      <xs:element name="DisplayBonusBeforePurchase" type="EmptyElement" minOccurs="0"
maxOccurs="1"/>
      <xs:element name="FeedbackActivationConditions" type="PerkDataConditions" minOccurs="0"
maxOccurs="1"/>
    </xs:all>
  </xs:complexType>
</xs:element>
```

Child elements

SkillsToShow

The SkillsToShow element is pretty self-explanatory: it is used to display one or several skill(s) tooltip(s) when hovering the perk in the perk tree. This is used for perks like HumanBallista, resupply, etc... You can also specify an OverallUses if you want the skill to have a define amount of uses per night.

```
<xs:complexType name="SkillsToShow">
  <xs:sequence>
    <xs:element name="SkillToShow">
      <xs:complexType>
        <xs:complexContent>
          <xs:extension base="EmptyElement">
            <xs:attribute name="Id" type="xs:string" use="required"/>
            <xs:attribute name="OverallUses" type="xs:integer" use="optional"/>
          </xs:extension>
        </xs:complexContent>
      </xs:complexType>
    </xs:element>
  </xs:sequence>
</xs:complexType>
```



CompendiumEntries

CompendiumEntries are used to display tooltips that explain some mechanics of the game. For example, "SkillEffect_IgnoreLineOfSight" shows the explanation of momentum the same way the game does when you hover a skill that has this skill effect. The same thing goes for "DamageType_Physical" or "GameConcept_Wounds". This is not necessary when showing a skill that already has this effect, as the skill tooltip will automatically show its skill effects informations.

```
<xs:complexType name="CompendiumEntries">
  <xs:sequence>
    <xs:element name="CompendiumEntry" type="CompendiumEntry" minOccurs="1"
maxOccurs="unbounded"/>
  </xs:sequence>
</xs:complexType>

<xs:complexType name="CompendiumEntry">
  <xs:attribute name="Id" type="xs:token" use="required"/>
  <xs:attribute name="DisplayLinkedEntries" type="xs:boolean" use="optional"/>
</xs:complexType>
```



DisplayInHUD

This element contains every information for the HUD behavior. Each element is fairly simple:

Bonus has a single Value attribute. This value is being printed as the green number at the bottom right of the perk display.	
Buffer has a single Value attribute. This value is being printed as the white number located under the perk display.	
GreyOut is a list of Conditions (see Conditions). If all conditions are true, the perk will appear greyed out in the HUD.	
Highlight is a list of Conditions (see Conditions). If all conditions are true, the perk will be highlighted, both at the bottom screen and right screen location.	
HoverDisplay has a simple range attribute. It is used to display tiles around the perk owner on hover, in order to show the perk range. Multiple HoverRange can be present, as in the LoneWolf perk.	

```
<xs:complexType name="DisplayInHUD">
  <xs:all>
    <xs:element name="Bonus" type="ValueTokenAttribute" minOccurs="0" maxOccurs="1"/>
    <xs:element name="Buffer" type="ValueTokenAttribute" minOccurs="0" maxOccurs="1"/>
    <xs:element name="GreyOut" type="PerkDataConditions" minOccurs="0" maxOccurs="1"/>
    <xs:element name="Highlight" type="PerkDataConditions" minOccurs="0" maxOccurs="1"/>
    <xs:element name="HoverDisplay" type="HoverDisplay" minOccurs="0" maxOccurs="1"/>
  </xs:all>
</xs:complexType>
```

```
<xs:complexType name="HoverDisplay">
  <xs:sequence>
    <xs:element name="HoverRange" minOccurs="1" maxOccurs="unbounded">
      <xs:complexType>
        <xs:attribute name="Range" type="xs:token" use="required"/>
      </xs:complexType>
    </xs:element>
  </xs:sequence>
</xs:complexType>
```


DisplayBonusBeforePurchase

This simple element applies to perks that has a PerkEffectInformations localization key. This localization key is used to give information on the current state or bonus given by the perk. Sometimes, we want to display to potential bonus before the purchase of the perk. This element is used to do so. Example with Bodybuilder:



FeedbackActivationConditions

This element is a list of Conditions (see Conditions) that are used by the RefreshPerkActivationFeedback action. This define if the perk display should be displayed as newly activated (all the conditions are true) or newly deactivated (at least one condition is false).

Modules

Definition

Modules are the very heart of the perk system. They contain Conditions, Events and Effects.

Perk definitions must have at least one module.

Types of Modules

BufferModule

Definition

BufferModule contains 3 internal buffers, that can be interacted with through buffer module specific actions. It is very useful to store a bunch of information during runtime, such as the number of kill, or the amount of any resources spent. Though it is rarely used in our perk designs, the buffer values are saved upon quitting the game, meaning a perk can store buffer values for as long as you want.

Example

The perks Berserk and Adrenaline rush are a very straight forward example for the basic usage of buffers.

GaugeModule

Definition

GaugeModule inherits from BufferModule, meaning they also have access to buffers. This module is meant to be used for perks that acts like additional gauges, such as BackProtection and OrganicArmor.

Example

The perks BackProtection and OrganicArmor are a very straight forward example for the basic usage of GaugeModules.

Conditions

Definition

Conditions are used in modules, events and sometimes effects. Conditions are optional, and only one group can exist. In that group, there might be multiple sub-conditions. When that's the case, for the group to be valid, all the sub-conditions must be true.

Sub-conditions are checked from top to bottom. As soon as one condition is not valid, the verifications stops and the group is marked as invalid. This means that you should use securities (see Interpreter) at the top of your conditions, so if your securities aren't valid, the following conditions won't be verified.

Conditions are also evaluated only once, meaning if the module conditions were valid before an event was called, they will still be valid until we reached the end of the events calls, no matter how the module was modified by the current events. If you want to handle conditions event by event, you have to specify conditions inside of those events.

Types

There are only two sub-conditions available: IsTrue and IsFalse. Both require a Value attribute. They are made to be paired with the Interpreter, either using available bool data or calculus paired with the equal, superior, or different operators.

Example

Let's take the SteadyAim perk as an example:

Here, we are storing the total moves in the first buffer of a buffer module. In order for the perk to be active, its only condition is that this buffer must be equal to 0.

```
<Conditions>
  <IsTrue Value="Module.Buffer = 0"/>
</Conditions>
```

Actions

Definition

Actions are meant to access and edit the Module they are a part of. They are also used to trigger perk effects on a given event.

```
<xs:complexType name="Actions">
  <xs:sequence>
    <xs:choice minOccurs="0" maxOccurs="unbounded">
      <xs:element name="TriggerEffects" type="EmptyElement"/>
      <xs:element name="TriggerEffectsOnAllAttackData" type="EmptyElement"/>
      <xs:element name="SetBufferTo" type="SetBufferTo"/>
      <xs:element name="IncreaseBuffer" type="SetBufferTo"/>
      <xs:element name="DecreaseBuffer" type="SetBufferTo"/>
      <xs:element name="InstantiateStatEffectDisplay" type="InstantiateEffectDisplay"/>
      <xs:element name="InstantiateRestoreEffectDisplay" type="InstantiateRestoreEffectDisplay"/>
      <xs:element name="InstantiateBuffEffectDisplay" type="InstantiateBuffEffectDisplay"/>
      <xs:element name="RefreshPerkActivationFeedback" type="RefreshPerkActivationFeedback"/>
    </xs:choice>
  </xs:sequence>
</xs:complexType>
```

Types

TriggerEffects

This action is an empty element. It is used to trigger the effects with the current event data.

TriggerEffectsOnAllAttackData

This action is an empty element. It is used to trigger the effects on each target contained in the current data. We had to use this for the perk Boom, in order for the explosion to apply **after** the current skill execution instance.

SetBufferTo

This action is used to store a value inside a specified buffer. There are currently 3 available buffers. Use the BufferIndex attribute to choose which buffer to edit. This attribute is optional, as by default this action will modify the first buffer (Module.Buffer).

```
<xs:complexType name="SetBufferTo">
  <xs:attribute name="BufferIndex" type="BufferIndex" use="optional"/>
  <xs:attribute name="Value" type="xs:token" use="required"/>
</xs:complexType>
```

```
<xs:simpleType name="BufferIndex">
  <xs:restriction base="xs:token">
    <xs:enumeration value="Buffer"/>
    <xs:enumeration value="Buffer2"/>
    <xs:enumeration value="Buffer3"/>
  </xs:restriction>
</xs:simpleType>
```

IncreaseBuffer

This action follows the same scheme as the SetBufferTo action. Instead of replacing the value stored in the specified buffer, it increases it by the amount specified.

DecreaseBuffer

This action follows the same scheme as the SetBufferTo action. Instead of replacing the value stored in the specified buffer, it decreases it by the amount specified.

InstantiateStatEffectDisplay

This action is used to display feedback corresponding to the specified stat with the specified value.

```
<xs:complexType name="InstantiateEffectDisplay">
  <xs:attribute name="Stat" type="AllStats" use="required"/>
  <xs:attribute name="Value" type="xs:token" use="required"/>
</xs:complexType>
```

InstantiateRestoreEffectDisplay

This action is used to display feedback corresponding to the specified stat with the specified value, automatically adding the “Restore” icon.

```
<xs:complexType name="InstantiateRestoreEffectDisplay">
  <xs:attribute name="Stat" type="RegenerableStat" use="required"/>
  <xs:attribute name="Value" type="xs:token" use="required"/>
</xs:complexType>
```

InstantiateBuffEffectDisplay

This action is used to display feedback corresponding to the specified stat with the specified value, automatically adding the “Buff” icon.

```
<xs:complexType name="InstantiateBuffEffectDisplay">
  <xs:attribute name="Stat" type="ModifiableStat" use="required"/>
  <xs:attribute name="Value" type="xs:token" use="required"/>
</xs:complexType>
```

RefreshPerkActivationFeedback

This action is used to display the feedback regarding the stat of the perk. It is mandatory to have FeedbackActivationConditions defined in the perk.

If the RefreshView attribute is true, this will be the behavior: in the event where the perk went from inactive to active, it will print “+ Perk name” in game. This also works in the other way. If the state didn’t change, it won’t print anything.

Else, if the RefreshView attribute is false, it is only useful to “save” the state of the perk for the next feedback, as no feedback will be instantiated.

```
<xs:complexType name="RefreshPerkActivationFeedback">
  <xs:complexContent>
    <xs:extension base="EmptyElement">
      <xs:attribute name="RefreshView" type="xs:boolean" use="optional" />
    </xs:extension>
  </xs:complexContent>
</xs:complexType>
```

Events

Definition

Events correspond to actions that occurs regularly in game. Some are linked to the perk owner itself (like OnTargetKilled or OnHitTaken) whereas some are global events (like OnEnemyMovementEnd). Same events in modules are called in order, meaning if you have multiple OnMovementEnd events, they will be called from top to bottom, so the modifications you made in the first occurrence of the event handler will be applied before entering the second event handler. This logic is sometimes mandatory, as you might want to call different actions depending on conditions within the same event.

Events contains at least one action and can contain conditions as well. If any condition is not true, the actions won't be called.

Types

Event name	Linked to	Description
None	-	NOT RELEVANT TO PERKS
Permanent	-	NOT RELEVANT TO PERKS
OnDeath	Owner	Called when the owner dies
OnCreation	-	NOT RELEVANT TO PERKS
OnTileCrossed	Owner	Called each time the owner crosses a tile while moving.
OnMovementEnd	Owner	Called at the end of a movement, including teleports and maneuver.
OnHitTaken	Owner	Called when the owner receives a hit from a skill that is an attack (meaning not dodged).
OnGenericHitTaken	Owner	Called when the owner receives a hit (meaning not dodged).
OnDodge	Owner	Called when the owner dodges.
OnTargetHit	Owner	Called each time the owner hit a target, meaning it will be called for each target (enemy, building, allies..) damaged by an attack skill.
OnTargetDodge	Owner	Called each time the owner misses a target, meaning it will be called for each target (enemy, building, allies..) dodging an attack skill.
OnTargetKilled	Owner	Called each time the owner kills a target, meaning it will be called for each target (enemy, building, allies...) killed by an attack skill.
OnSkillNextHit	Owner	Called between each hit of a skill cast by the owner. Will always be called after the OnSkillCastBegin event and before the OnSkillCastEnd. Even if the skill is not "multi-hit skill", it will still be called once. NEVER CALLED WHEN THE SKILL EXECUTION COMES FROM A PERK.

OnSkillCastBegin	Owner	Called at the very beginning of a skill being cast by the owner. NEVER CALLED WHEN THE SKILL EXECUTION COMES FROM A PERK.
OnSkillCastEnd	Owner	Called just at the end of a skill being cast by the owner. NEVER CALLED WHEN THE SKILL EXECUTION COMES FROM A PERK.
OnSkillStatusApplied	Owner	Called when the owner applies a status through the execution of a skill.
OnStatusApplied	Owner	Called when the owner applies a status.
OnPoisonProc	-	Called each time a poison applies its damages.
OnEnemyMovementEnd	-	Called when the enemies finished their movement phase.
OnBehaviorClusterExecutionEnd	-	Called at the end of each enemy cluster execution (just before the camera moves from one attack group to another).
OnDefenseTargetKilled	-	Called each time a target is killed by a building
OnStartTurn	-	Called at each start of turns (day, night, playable, enemy...)
OnStartNightTurnPlayable	-	Called at the start of the player night turn.
OnStartNightTurnEnemy	-	Called at the start of the enemies' night turn.
OnStartProductionTurn	-	Called at the start of the production turn.
OnStartDeploymentTurn	-	Called at the start of the deployment turn.
OnEndTurn	-	Called at each end of turns (day, night, playable, enemy...)
OnEndNightTurnPlayable	-	Called at the end of the player night turn.
OnEndNightTurnEnemy	-	Called at the end of the enemies' night turn.
OnEndProductionTurn	-	Called at the end of the production turn.
OnEndDeploymentTurn	-	Called at the end of the deployment turn.
OnCreationAfterViewInitialized	-	Called upon the perk unlock

Example

Mana shield perk

```
<Event EffectTime="OnSkillCastEnd">
  <Conditions>
    <IsTrue Value="ActivationCondition"/>
  </Conditions>
  <Actions>
    <IncreaseBuffer Value="Skill.ManaCost"/>
    <TriggerEffects/>
  </Actions>
</Event>
```

At the end of the execution of a skill cast by the perk owner, we make sure the skill has a mana cost (see TokenVariables and Conditions). If our conditions are all true, we store this mana cost into our buffer (see BufferModule and Actions). Once done, we trigger the effects contained in the perk (in our case, we apply a buff to Block using the value stored in the module's buffer, and we empty this buffer).

Effects

Definition

Effects are the only elements in the perk definition that have an impact in game. They are therefore the heart of the module. Some effects are permanent and do not need to be triggered, like StatModifiers or SkillModifiers. But some, like a CastSkill or ApplyStatus need to be triggered manually by an action linked to an event.

For all perk effects: the values are not of type int but are instead tokens. This is important for 2 things: TokenVariables and the Interpreter. This allows us to have dynamic values thanks to the data and the tools we have at our disposal.

PERMANENT EFFECTS are always active, as long as the Conditions for both the module and the effect are met.

TRIGGERABLE EFFECTS on the other hand need to be triggered via the TriggerEffects action. This will still make sure both the module and the effect conditions are met.

Types

AddSkillEffect

PERMANENT EFFECT

AddSkillEffect is used to add a skill effect to the skills of the perk owner. This is used for the HeadOn and ThirdEye perks.

```
<xs:complexType name="AddSkillEffect">
  <xs:all>
    <xs:element name="Conditions" type="PerkDataConditions" minOccurs="0" maxOccurs="1"/>
    <xs:element name="SkillEffects" type="AttackSkillEffects" minOccurs="1" maxOccurs="1"/>
  </xs:all>
</xs:complexType>
```

See AttackSkillEffects for the list of available skill effects.

AllowDiagonalPropagation

PERMANENT EFFECT

This perk effect has no settings besides conditions. Using this perk effect will allow diagonal propagation.

```
<xs:complexType name="AllowDiagonalPropagation">
  <xs:all>
    <xs:element name="Conditions" type="PerkDataConditions" minOccurs="0" maxOccurs="1"/>
  </xs:all>
</xs:complexType>
```

ApplyStatus

TRIGGERABLE EFFECT

This perk effect will apply a status to the target(s) you specify (see PerkTargeting). Everything else is pretty straight forward.

```
<xs:complexType name="ApplyStatus">
  <xs:sequence>
    <xs:element name="PerkTargeting" type="PerkTargeting" minOccurs="0" maxOccurs="1"/>
  </xs:sequence>
  <xs:attribute name="Status" type="Status" use="required"/>
  <xs:attribute name="Stat" type="ModifiableStat" use="optional"/>
  <xs:attribute name="TurnsCount" type="xs:token" use="required"/>
  <xs:attribute name="Value" type="xs:token" use="optional"/>
  <xs:attribute name="Chance" type="xs:token" use="optional" default="100"/>
  <xs:attribute name="HideDisplayEffect" type="xs:boolean" use="optional"/>
  <xs:attribute name="RefreshHUD" type="xs:boolean" use="optional" default="true"/>
</xs:complexType>
```

CastSkill

TRIGGERABLE EFFECT

This perk effect will cast a skill on the target(s) you specify (see PerkTargeting). The owner of the perk does **not** need to have the skill equipped/unlocked; this perk effect will dynamically load the skill based on the id you specified.

```
<xs:complexType name="CastSkill">
  <xs:sequence>
    <xs:element name="PerkTargeting" type="PerkTargeting" minOccurs="0" maxOccurs="1"/>
  </xs:sequence>
  <xs:attribute name="SkillId" type="xs:token" use="required"/>
</xs:complexType>
```

ComputationStatLocker

PERMANENT EFFECT

This perk effect will lock the value of the specified skill computation stat to 0. This was especially useful for the BloodMagic perk, locking the mana cost for all skills computation to 0. See SkillComputationStat for the list of stats.

```
<xs:complexType name="ComputationStatLocker">
  <xs:attribute name="ComputationStat" type="SkillComputationStat" use="required"/>
</xs:complexType>
```

DealDamage

TRIGGERABLE EFFECT

This perk effect will deal damages to the target(s) you specify (see PerkTargeting). The damage dealt have no damage type, meaning it won't be increase by the stats of the perk owner. If IgnoreDefense is set to true, the damages dealt will by-pass the block and resistance of the target(s). It will still hit the armor though.

```
<xs:complexType name="DealDamage">
  <xs:sequence>
    <xs:element name="PerkTargeting" type="PerkTargeting" minOccurs="0" maxOccurs="1"/>
  </xs:sequence>
  <xs:attribute name="IgnoreDefense" type="xs:boolean" use="optional"/>
  <xs:attribute name="Value" type="xs:token" use="required"/>
</xs:complexType>
```

DynamicStatsModifier

PERMANENT EFFECT

This perk effect was made for a very specific use. It is only used by the Versatility perk, as it works in a very unique way. This perk effect will take the value of the highest stat you specified, and add this value as a bonus to all the other stat. The attribute value is used as a percentage.

```
<xs:complexType name="DynamicStatsModifier">
  <xs:sequence>
    <xs:element name="Stat" type="ModifiableStat" minOccurs="2" maxOccurs="unbounded"/>
  </xs:sequence>
  <xs:attribute name="Value" type="xs:token" use="required"/>
</xs:complexType>
```

EquipmentSlotModifier

PERMANENT EFFECT

This perk effect is used to add / remove a specified amount of slot, corresponding to the SlotId you specified. A positive value will add an amount, a negative value will remove an amount.

```
<xs:complexType name="EquipmentSlotModifier">
  <xs:attribute name="SlotId" type="SlotId" use="required"/>
  <xs:attribute name="Value" type="xs:token" use="required"/>
</xs:complexType>
```

GetAdditionalExperience

TRIGGERABLE EFFECT

This perk effect will add the value you specified into the total experience gained by the perk owner during the current night. It will not give the experience right away since the units only collect it at the end of a night.

```
<xs:complexType name="GetAdditionalExperience">
  <xs:attribute name="Value" type="xs:token" use="required"/>
</xs:complexType>
```

ModifyDefenseDamage

PERMANENT EFFECT

This perk effect will increase the damage of the defenses (traps, ballistae, catapults...) in the range you specified by the percentage you specified. These perk effects **can** be stacked.

```
<xs:complexType name="ModifyDefensesDamage">
  <xs:attribute name="Range" type="xs:token" use="required"/>
  <xs:attribute name="Percentage" type="xs:token" use="required"/>
</xs:complexType>
```

ResetBuffer

TRIGGERABLE EFFECT

This perk effect does not have any impact in game. It is basically a shortcut to the SetBufferTo action. This was meant to be used to centralize the reset of buffer value, in the case of a perk that has different triggering events. It only reset the first buffer (meaning Module.Buffer) and if no value is specified, it will reset the buffer to the DefaultValue that was specified in the BufferModule definition.

```
<xs:complexType name="ResetBuffer">
  <xs:attribute name="ResetValue" type="xs:integer" use="optional"/>
</xs:complexType>
```

RestoreStat

TRIGGERABLE EFFECT

This perk effect is used to restore stats that can be restored (see RegenerableStat). The value specified **will** be modified by the stat's receiving modifier (only applies to Health with the HealingReceived stat).

```
<xs:complexType name="RestoreStat">
  <xs:attribute name="Stat" type="RegenerableStat" use="required"/>
  <xs:attribute name="Value" type="xs:token" use="required"/>
  <xs:attribute name="HideDisplayEffect" type="xs:boolean" use="optional"/>
</xs:complexType>
```

SkillModifier

PERMANENT EFFECT

This perk effect will act like a bonus or malus to the skills of the perk owner. See SkillComputationStat for the complete list of stat modifiable. The values are all additives. Flat damage acts like a raw +x, whereas Overall damage increases your overall damage stat, acting like a multiplier on the base damage of the skill.

```
<xs:complexType name="SkillModifier">
  <xs:all>
    <xs:element name="Conditions" type="PerkDataConditions" minOccurs="0" maxOccurs="1"/>
  </xs:all>
  <xs:attribute name="ComputationStat" type="SkillComputationStat" use="required"/>
  <xs:attribute name="Value" type="xs:token" use="required"/>
</xs:complexType>
```

StatLocker

PERMANENT EFFECT

This perk effect is used to lock a stat to 0. Locked stats are still usable in interpreter with their “Unlocked” counterpart. Example: Organic armor can still access the total armor value via Owner.UnlockedArmorTotal. This means Organic armor benefits every bonus the owner has on its ArmorTotal stat, but the ArmorTotal value in game uses the locked value.

```
<xs:complexType name="StatLocker">
  <xs:attribute name="Stat" type="ModifiableStat" use="required"/>
</xs:complexType>
```

StatModifier

PERMANENT EFFECT

This perk effect will act like a bonus or malus to the specified stat for the perk owner. See ModifiableStat for the complete list. The ChildStatFollows attribute define the behavior of the perk when the value is evolving. If the attribute is set to true, the perk owner “child stat” will move with its parent value. This happens for the nimble perk, where it is set to false: if the nimble is activated during the night, the player will gain bonus MovePointsTotal, but its current MovePoints will not be increased.

```
<xs:complexType name="StatModifier">
  <xs:attribute name="Stat" type="ModifiableStat" use="required"/>
  <xs:attribute name="Value" type="xs:token" use="required"/>
  <xs:attribute name="ChildStatFollows" type="xs:boolean" use="optional"/>
</xs:complexType>
```

SwapContextualSkill

PERMANENT EFFECT

On perk unlock, this perk effect will replace the specified contextual skill by the other specified contextual skill. If no OverallUses is specified, the skill will have an infinite amount of uses per night. The uses per turn are bound to the skill itself.

```
<xs:complexType name="SwapContextualSkill">
  <xs:attribute name="ContextualSkillIdToLock" type="xs:token" use="required"/>
  <xs:attribute name="ContextualSkillIdToUnlock" type="xs:token" use="required"/>
  <xs:attribute name="OverallUses" type="xs:positiveInteger" use="optional"/>
</xs:complexType>
```

UnlockContextualSkill

PERMANENT EFFECT

On perk unlock, this perk effect will unlock the specified contextual skill. If no OverallUses is specified, the skill will have an infinite amount of uses per night. The uses per turn are bound to the skill itself.

```
<xs:complexType name="UnlockContextualSkill">
  <xs:attribute name="ContextualSkillId" type="xs:token" use="required"/>
  <xs:attribute name="OverallUses" type="xs:positiveInteger" use="optional"/>
</xs:complexType>
```

Miscellaneous

AttackSkillEffects

This element represents every skill effect existing in the game. You basically add any skill effect you want to the owner skills with the AddSkillEffect effect.

```
<xs:complexType name="AttackSkillEffects">
  <xs:choice maxOccurs="unbounded">
    <xs:element name="ArmorPiercing" minOccurs="0"/>
    <xs:element name="ArmorShredding" minOccurs="0">
      <xs:complexType>
        <xs:attribute name="BonusDamage" type="xs:float" default="1" use="optional" />
      </xs:complexType>
    </xs:element>
    <xs:element name="Buff" type="SkillEffectBuff" minOccurs="0" maxOccurs="unbounded"/>
    <xs:element name="Debuff" type="SkillEffectDebuff" minOccurs="0" maxOccurs="unbounded"/>
    <xs:element name="ExileCaster" minOccurs="0"/>
    <xs:element name="Follow" minOccurs="0"/>
    <xs:element name="IgnoreLineOfSight" minOccurs="0" maxOccurs="1"/>
    <xs:element name="Inaccurate" type="SkillEffectInaccurate" minOccurs="0"/>
    <xs:element name="Isolated" type="SkillEffectIsolated" minOccurs="0"/>
    <xs:element name="Maneuver" minOccurs="0"/>
    <xs:element name="Momentum" type="SkillEffectMomentum" minOccurs="0"/>
    <xs:element name="MultiHit" type="SkillEffectMultiHit" minOccurs="0"/>
    <xs:element name="NoBlock" minOccurs="0"/>
    <xs:element name="NoDodge" minOccurs="0"/>
    <xs:element name="Opportunistic" type="SkillEffectOpportunistic" minOccurs="0"/>
    <xs:element name="Poison" type="SkillEffectPoison" minOccurs="0"/>
    <xs:element name="Propagation" type="SkillEffectPropagation" minOccurs="0"/>
    <xs:element name="DecreaseStat" type="SkillEffectDecreaseStat" minOccurs="0"/>
    <xs:element name="RegenStat" type="SkillEffectRegenStat" minOccurs="0"/>
    <xs:element name="Stun" type="SkillEffectStun" minOccurs="0"/>
    <xs:element name="Contagion" type="SkillEffectContagion" minOccurs="0"/>
    <xs:element name="Charged" type="SkillEffectCharged" minOccurs="0" maxOccurs="unbounded"/>
    <xs:element name="RemoveStatus" type="RemoveStatus" minOccurs="0" maxOccurs="unbounded"/>
    <xs:element name="SurroundingEffect" type="SkillEffectSurroundingEffect" minOccurs="0"/>
    <xs:element name="CasterEffect" type="SkillEffectCasterEffect" minOccurs="0"/>
  </xs:choice>
</xs:complexType>
```

ModifiableStat

ModifiableStat is pretty self-explanatory, it represents every existing modifiable stat in the game.

```
<xs:simpleType name="ModifiableStat">
  <xs:restriction base="xs:string">
    <xs:enumeration value="HealthTotal"/>
    <xs:enumeration value="ManaTotal"/>
    <xs:enumeration value="ActionPointsTotal"/>
    <xs:enumeration value="MovePointsTotal"/>
    <xs:enumeration value="ArmorTotal"/>
    <xs:enumeration value="Dodge"/>
    <xs:enumeration value="Block"/>
    <xs:enumeration value="Resistance"/>
    <xs:enumeration value="PhysicalDamage"/>
    <xs:enumeration value="MagicalDamage"/>
    <xs:enumeration value="RangedDamage"/>
    <xs:enumeration value="BonusDamageMultiplier"/>
    <xs:enumeration value="Critical"/>
    <xs:enumeration value="HealthRegen"/>
    <xs:enumeration value="ManaRegen"/>
    <xs:enumeration value="CriticalPower"/>
    <xs:enumeration value="HealingReceived"/>
    <xs:enumeration value="SkillRangeModifier"/>
    <xs:enumeration value="ExperienceGainMultiplier"/>
    <xs:enumeration value="OverallDamage"/>
    <xs:enumeration value="Reliability"/>
    <xs:enumeration value="Accuracy"/>
    <xs:enumeration value="StunChanceModifier"/>
    <xs:enumeration value="PropagationBouncesModifier"/>
    <xs:enumeration value="PropagationDamage"/>
    <xs:enumeration value="DodgeReduction"/>
    <xs:enumeration value="ResistanceReduction"/>
    <xs:enumeration value="MomentumAttacks"/>
    <xs:enumeration value="OpportunisticAttacks"/>
    <xs:enumeration value="IsolatedAttacks"/>
    <xs:enumeration value="PoisonDamageModifier"/>
    <xs:enumeration value="MultiHitsCountModifier"/>
    <xs:enumeration value="PoisonDurationModifier"/>
    <xs:enumeration value="DebuffDurationModifier"/>
    <xs:enumeration value="BuffDurationModifier"/>
    <xs:enumeration value="PotionRangeModifier"/>
    <xs:enumeration value="MagicDamageReductionModifier"/>
    <xs:enumeration value="BonusSkillUses"/>
    <xs:enumeration value="BonusUsableItemsUses"/>
    <xs:enumeration value="StunDurationModifier"/>
    <xs:enumeration value="ContagionDurationModifier"/>
  </xs:restriction>
</xs:simpleType>
```

RegenerableStat

RegenerableStat is pretty self-explanatory, it represents every existing regenerable stats in the game.

```
<xs:simpleType name="RegenerableStat">
  <xs:restriction base="xs:string">
    <xs:enumeration value="Health"/>
    <xs:enumeration value="Mana"/>
    <xs:enumeration value="ActionPoints"/>
    <xs:enumeration value="MovePoints"/>
  </xs:restriction>
</xs:simpleType>
```

SkillComputationStat

SkillComputationStat represent any variables taken in count during the computation of a skill. One important thing to have in mind: all cost elements count as a **reduction**, therefore a positive value means a reduction of the cost, whereas a negative value acts like an augmentation of the cost. Besides FlatDamage, everything else is added to the already existing stats for the perk owner.

```
<xs:simpleType name="SkillComputationStat">
  <xs:restriction base="xs:string">
    <xs:enumeration value="Critical"/>
    <xs:enumeration value="CriticalPower"/>
    <xs:enumeration value="OverallDamage"/>
    <xs:enumeration value="Reliability"/>
    <xs:enumeration value="Accuracy"/>
    <xs:enumeration value="MomentumAttacks"/>
    <xs:enumeration value="OpportunisticAttacks"/>
    <xs:enumeration value="IsolatedAttacks"/>
    <xs:enumeration value="PoisonDamageModifier"/>
    <!-- COSTS -->
    <xs:enumeration value="ActionPointsCost"/>
    <xs:enumeration value="ManaCost"/>
    <xs:enumeration value="HealthCost"/>
    <xs:enumeration value="MovePointsCost"/>
    <!-- REDUCTION -->
    <xs:enumeration value="MagicResistanceReduction"/>
    <xs:enumeration value="MeleeArmorShreddingBonus"/>
    <!-- OTHER -->
    <xs:enumeration value="FlatDamage"/>
  </xs:restriction>
</xs:simpleType>
```


PerkTargeting

This element is used by most of the perks that need to design some targets, like SpikySkin, HumanBallista, Boom, or anything that applies a status like Poisonous for example.

```
<xs:complexType name="PerkTargeting">
  <xs:sequence>
    <xs:element name="DamageableTarget" type="DamageableTarget" minOccurs="0"
maxOccurs="unbounded"/>
  </xs:sequence>
  <xs:attribute name="TargetingReference" type="TargetingReference" use="required"/>
  <xs:attribute name="TargetingMethod" type="TargetingMethod" use="required"/>
  <xs:attribute name="Amount" type="xs:token" use="optional"/>
  <xs:attribute name="Range" type="xs:token" use="optional"/>
</xs:complexType>
```

DamageableTarget are optional. By default, the targeting does not look at the type of the target and takes every target found. If you specify at least one DamageableType, it will only affect those types (for example, Poisonous only targets enemies and bosses).

```
<xs:complexType name="DamageableTarget">
  <xs:attribute name="Type" type="DamageableType"/>
</xs:complexType>
```

```
<xs:simpleType name="DamageableType">
  <xs:restriction base="xs:token">
    <xs:enumeration value="Playable"/>
    <xs:enumeration value="Enemy"/>
    <xs:enumeration value="Boss"/>
    <xs:enumeration value="Building"/>
    <xs:enumeration value="Obstacle"/>
    <xs:enumeration value="Other"/>
  </xs:restriction>
</xs:simpleType>
```

The TargetingReference element is very important and therefore mandatory. It basically defines the source for the following TargetingMethod element. Owner and Caster may be different, depending on the source event.

Owner correspond to the perk owner, Caster correspond to the caster of the potential skill that triggered the effect, Target correspond to the target of the potential skill that triggered the effect (meaning it can be the Owner if this is coming from a OnHitTaken event!) and AllTargets correspond to every target that were hit by the potential skill that triggered the effect (therefore, usually used with a OnSkillCastEnd event).

```
<xs:simpleType name="TargetingReference">
  <xs:restriction base="xs:token">
    <xs:enumeration value="Owner"/>
    <xs:enumeration value="Caster"/>
    <xs:enumeration value="Target"/>
    <xs:enumeration value="AllTargets"/>
  </xs:restriction>
</xs:simpleType>
```

The TargetingMethod element is also very important and therefore mandatory. It is entirely dependent on the reference specified earlier on.

Self will target the reference itself, whereas the ClosestTarget, AdjacentDamageables and DamageablesInRange will look **around** the reference and use the Amount and Range attributes to make their choice. The absence of Amount means infinite, whereas the absence of Range means a range of 1.

```
<xs:simpleType name="TargetingMethod">
  <xs:restriction base="xs:token">
    <xs:enumeration value="Self"/>
    <xs:enumeration value="ClosestTarget"/>
    <xs:enumeration value="AdjacentDamageables"/>
    <xs:enumeration value="DamageablesInRange"/>
  </xs:restriction>
</xs:simpleType>
```

Status

Status is pretty self-explanatory; it represents every existing status in the game.

```
<xs:simpleType name="Status">
  <xs:restriction base="xs:token">
    <xs:enumeration value="Buff"/>
    <xs:enumeration value="Debuff"/>
    <xs:enumeration value="Poison"/>
    <xs:enumeration value="Stun"/>
    <xs:enumeration value="Charged"/>
    <xs:enumeration value="Contagion"/>
  </xs:restriction>
</xs:simpleType>
```

SlotId

SlotId is pretty self-explanatory; it represents every existing type of slot in the game.

```
<xs:simpleType name="SlotId">
  <xs:restriction base="xs:string">
    <xs:enumeration value="None"/>
    <xs:enumeration value="RightHand"/>
    <xs:enumeration value="LeftHand"/>
    <xs:enumeration value="Body"/>
    <xs:enumeration value="Foot"/>
    <xs:enumeration value="Head"/>
    <xs:enumeration value="Trinket"/>
    <xs:enumeration value="Usables"/>
    <xs:enumeration value="Inventory"/>
    <xs:enumeration value="Shop"/>
    <xs:enumeration value="RewardItem"/>
  </xs:restriction>
</xs:simpleType>
```

Examples: Creation of a perk step by step

Introduction

Disclaimer: if you went there directly without reading the details about each perk elements, you might not understand every subject discussed. I will not give details about each action, effects, events... I will be using, but I'll still link corresponding information from time to time.

My goal here is to show multiple existing perks, from very easy to hard or very specific, in order to give an exhaustive list of possibilities and to give you the tools to do the same by your own.

Level: very easy – GlassCannon



First Step: Id and variables

The very first step in a creation of a perk is to think about exactly what we want to do. In this first example, we will recreate the Glass Cannon perk.

I like to start my perk integration directly by defining my “variables” using TokenVariables. Here, we need to define both our bonus and malus values:

```
<PerkDefinition Id="GlassCannon">
  <TokenVariables>
    <TokenVariable Key="HealthTotalBonus" Value="-60"/>
    <TokenVariable Key="CriticalBonus" Value="15"/>
  </TokenVariables>
</PerkDefinition>
```

Second Step: Modules

Once our values are defined, we can start to use them in the perk logic. In this example, no events or conditions are needed, as the perk acts like a permanent buff and debuff. Therefore, we are going straight to the goal by defining two StatModifier effects.

As you can see, I made sure to use the values I defined earlier in the TokenVariables for the values of our StatModifiers. Both the keys will be replaced by their respective value.

```
<PerkDefinition Id="GlassCannon">
  <TokenVariables>
    <TokenVariable Key="HealthTotalBonus" Value="-60"/>
    <TokenVariable Key="CriticalBonus" Value="15"/>
  </TokenVariables>
  <Modules>
    <BufferModule>
      <Effects>
        <StatModifier Stat="HealthTotal" Value="HealthTotalBonus"/>
        <StatModifier Stat="Critical" Value="CriticalBonus"/>
      </Effects>
    </BufferModule>
  </Modules>
</PerkDefinition>
```

And with that, we already have a working perk.

Third Step: Localization

Again, the description of the Glass Cannon perk is very straight forward. We only need to show how the perk affects two stats. Luckily for us, the StatArgument does everything automatically for us. Therefore, we only need to format the perk description using two parameters like so:

KEY	VALUE
PerkDescription_GlassCannon	{0}. {1}.

Now, in order to replace those parameters with our value, we are going to define two StatArgument, using once again our variables that we defined in our TokenVariables earlier on.

```
<PerkDefinition Id="GlassCannon">
  <TokenVariables>
    <TokenVariable Key="HealthTotalBonus" Value="-60"/>
    <TokenVariable Key="CriticalBonus" Value="15"/>
  </TokenVariables>
  <LocArguments>
    <StatArgument Stat="HealthTotal" Value="HealthTotalBonus"/>
    <StatArgument Stat="Critical" Value="CriticalBonus"/>
  </LocArguments>
  <Modules>
    <BufferModule>
      <Effects>
        <StatModifier Stat="HealthTotal" Value="HealthTotalBonus"/>
        <StatModifier Stat="Critical" Value="CriticalBonus"/>
      </Effects>
    </BufferModule>
  </Modules>
</PerkDefinition>
```

And there we have it: the completed definition of the Glass Cannon perk. This kind of perk is obviously very easy to implement. It might look overkill to use LocArguments and TokenVariables for such a small perk but in reality, it makes it very easy to maintain. If we want to change one value or one of the affected stats, we only need to change it in the TokenVariables, and the entire perk will be up to date.

Level: Fairly easy – NightOwl



First Step: Id and variables

As usual, we want to split our variables as much as possible, since we will probably need to access every part of information regarding the perk later on, both in the view or in the Localization itself.

Here, the very first variables we define are **NightThreshold**, **MovePointsBonus** and **ActionPointsBonus**. They are the heart of the perk and will be used for sure in the Localization to describe the perk.

Then, we define two variables:

- **TurnsLeftToTrigger** which equals to the remaining amount of turns left for the perk to trigger.
- **TriggerCondition** which checks if **TurnsLeftToTrigger** equals 0, meaning the perk has met its threshold.

```
<PerkDefinition Id="NightOwl">
  <TokenVariables>
    <TokenVariable Key="TriggerCondition" Value="TurnsLeftToTrigger = 0"/>
    <TokenVariable Key="TurnsLeftToTrigger" Value="Max(0, NightThreshold - CurrentNightHour)"/>
    <TokenVariable Key="ActionPointsBonus" Value="2"/>
    <TokenVariable Key="MovePointsBonus" Value="3"/>
    <TokenVariable Key="NightThreshold" Value="5"/>
  </TokenVariables>
</PerkDefinition>
```

Once this is done, we do not need any other information, so we can start implementing the gameplay behavior.

Second Step: Modules

Effects

We want our perk to increase the total amount of actions points by our defined amount, as well as the total amount of move points by our other defined amount.

To do so, we simply need to define two StatModifier effects, both having their respective stat and value, like so:

```

<PerkDefinition Id="NightOwl">
  <TokenVariables>
    <TokenVariable Key="TriggerCondition" Value="TurnsLeftToTrigger = 0"/>
    <TokenVariable Key="TurnsLeftToTrigger" Value="Max(0, NightThreshold - CurrentNightHour)"/>
    <TokenVariable Key="ActionPointsBonus" Value="2"/>
    <TokenVariable Key="MovePointsBonus" Value="3"/>
    <TokenVariable Key="NightThreshold" Value="5"/>
  </TokenVariables>
  <Modules>
    <BufferModule>
      <Effects>
        <StatModifier Stat="ActionPointsTotal" Value="ActionPointsBonus"/>
        <StatModifier Stat="MovePointsTotal" Value="MovePointsBonus"/>
      </Effects>
    </BufferModule>
  </Modules>
</PerkDefinition>

```

Once this is done, we have a perk that gives bonuses permanently. This is not what we want, so we're going to add Conditions for the perk to be inactive until the threshold is reached.

Conditions

In the case of our perk, the condition for it to be active is very straight forward, as we already defined it in our TokenVariables.

```

<PerkDefinition Id="NightOwl">
  <TokenVariables>
    <TokenVariable Key="TriggerCondition" Value="TurnsLeftToTrigger = 0"/>
    <TokenVariable Key="TurnsLeftToTrigger" Value="Max(0, NightThreshold - CurrentNightHour)"/>
    <TokenVariable Key="ActionPointsBonus" Value="2"/>
    <TokenVariable Key="MovePointsBonus" Value="3"/>
    <TokenVariable Key="NightThreshold" Value="5"/>
  </TokenVariables>
  <Modules>
    <BufferModule>
      <Conditions>
        <IsTrue Value="TriggerCondition"/>
      </Conditions>
      <Effects>
        <StatModifier Stat="ActionPointsTotal" Value="ActionPointsBonus"/>
        <StatModifier Stat="MovePointsTotal" Value="MovePointsBonus"/>
      </Effects>
    </BufferModule>
  </Modules>
</PerkDefinition>

```

Now that we added the IsTrue condition, our effects won't be active until the TriggerCondition is true. Since TriggerCondition automatically get upon game initialization, our perk won't be active until the CurrentNightHour is superior or equal to our NightThreshold.

Events - clarity purposes

We already have a fully working perk at this point. Nevertheless, some steps are still missing in order to give more information to the player. SkillModifier are permanent effects, and as such, they do not trigger any visual feedback whatsoever.

In that regard, the last step for our Module is to add an event that allows us to indicate the player the perk became active. To do so, we'll instantiate two effect displays, one for each stat.

```
<PerkDefinition Id="NightOwl">
  <TokenVariables>
    <TokenVariable Key="TriggerCondition" Value="TurnsLeftToTrigger = 0"/>
    <TokenVariable Key="TurnsLeftToTrigger" Value="Max(0, NightThreshold - CurrentNightHour)"/>
    <TokenVariable Key="ActionPointsBonus" Value="2"/>
    <TokenVariable Key="MovePointsBonus" Value="3"/>
    <TokenVariable Key="NightThreshold" Value="5"/>
  </TokenVariables>
  <Modules>
    <BufferModule>
      <Events>
        <Event EffectTime="OnStartTurn">
          <Conditions>
            <IsFalse Value="IsEnemyTurn"/>
            <IsTrue Value="CurrentNightHour = NightThreshold"/> <!-- We don't use TriggerCondition directly
because it REMAINS true, and we only want to display this once -->
          </Conditions>
          <Actions>
            <InstantiateStatEffectDisplay Stat="ActionPoints" Value="ActionPointsBonus"/>
            <InstantiateStatEffectDisplay Stat="MovePoints" Value="MovePointsBonus"/>
          </Actions>
        </Event>
      </Events>
      <Conditions>
        <IsTrue Value="TriggerCondition"/>
      </Conditions>
      <Effects>
        <StatModifier Stat="ActionPointsTotal" Value="ActionPointsBonus"/>
        <StatModifier Stat="MovePointsTotal" Value="MovePointsBonus"/>
      </Effects>
    </BufferModule>
  </Modules>
</PerkDefinition>
```

We decided to display the feedback upon the start of the turn. Two conditions are needed:

- First, we want to make sure it is not the enemies' turn.
- Second, we want to check if the current night hour is equal to our threshold. We are not using the TriggerCondition variable as we only want to display the feedback once, when the threshold is met for the first time.

We then implemented two InstantiateStatEffectDisplay actions, according to our perk bonuses.

Third Step: View

The other very useful tool to give information to the player is the View element.

For the NightOwl perk, we want to do two very easy things:

- Grey out the perk when it is inactive, by using the GreyOut element alongside a IsFalse Condition.
- Indicate the remaining amount of turns left for the perk to be active, by using the TurnsLeftToTrigger variable in the Buffer element.

```
<PerkDefinition Id="NightOwl">
  <TokenVariables>
    <TokenVariable Key="TriggerCondition" Value="TurnsLeftToTrigger = 0"/>
    <TokenVariable Key="TurnsLeftToTrigger" Value="Max(0, NightThreshold - CurrentNightHour)"/>
    <TokenVariable Key="ActionPointsBonus" Value="2"/>
    <TokenVariable Key="MovePointsBonus" Value="3"/>
    <TokenVariable Key="NightThreshold" Value="5"/>
  </TokenVariables>
  <View>
    <DisplayInHUD>
      <GreyOut>
        <IsFalse Value="TriggerCondition" />
      </GreyOut>
      <Buffer Value="TurnsLeftToTrigger" />
    </DisplayInHUD>
  </View>
  <Modules>
    <BufferModule>
      <Events>
        <Event EffectTime="OnStartTurn">
          <Conditions>
            <IsFalse Value="IsEnemyTurn" />
            <IsTrue Value="CurrentNightHour = NightThreshold" />
          </Conditions>
          <Actions>
            <InstantiateStatEffectDisplay Stat="ActionPoints" Value="ActionPointsBonus" />
            <InstantiateStatEffectDisplay Stat="MovePoints" Value="MovePointsBonus" />
          </Actions>
        </Event>
      </Events>
      <Conditions>
        <IsTrue Value="TriggerCondition" />
      </Conditions>
      <Effects>
        <StatModifier Stat="ActionPointsTotal" Value="ActionPointsBonus" />
        <StatModifier Stat="MovePointsTotal" Value="MovePointsBonus" />
      </Effects>
    </BufferModule>
  </Modules>
</PerkDefinition>
```

Fourth Step: Localization

The final step is the Localization step. Since we were intelligent enough defining as much things as possible in separated variables, this will be very straight forward.

Here is the Localization keys and values for the NightOwl description:

KEY	VALUE
PerkDescription_NightOwl	Whenever the <style=KeyWord>{0}th turn</style> of the <style=Night>night</style> is reached: {1}. {2}. <style=Attenuated>This modifier is reset every morning.</style>
PerkEffectInformations_NightOwl	Turns left: {3}

We'll be using both LocArgument and StatArgument elements:

- LocArgument are useful for basic information, such as the threshold and the remaining turns.
- StatArgument since we are using StatModifier effects.

Also, you may have noticed that we are using a detailed description through the "PerkEffectInformations_NightOwl" localization value. This part of the description will only be displayed after the perk purchase.

Here are our 4 localization arguments, integrated in order:

```

<PerkDefinition Id="NightOwl">
  <TokenVariables>
    <TokenVariable Key="TriggerCondition" Value="TurnsLeftToTrigger = 0"/>
    <TokenVariable Key="TurnsLeftToTrigger" Value="Max(0, NightThreshold - CurrentNightHour)"/>
    <TokenVariable Key="ActionPointsBonus" Value="2"/>
    <TokenVariable Key="MovePointsBonus" Value="3"/>
    <TokenVariable Key="NightThreshold" Value="5"/>
  </TokenVariables>
  <LocArguments>
    <LocArgument Value="NightThreshold" Style="KeyWordNb"/>
    <StatArgument Stat="ActionPointsTotal" Value="ActionPointsBonus"/>
    <StatArgument Stat="MovePointsTotal" Value="MovePointsBonus"/>
    <LocArgument Value="TurnsLeftToTrigger" Interpreted="true" Style="KeyWordNb"/>
  </LocArguments>
  <View>
    <DisplayInHUD>
      <GreyOut>
        <IsFalse Value="TriggerCondition"/>
      </GreyOut>
      <Buffer Value="TurnsLeftToTrigger"/>
    </DisplayInHUD>
  </View>
  <Modules>
    <BufferModule>
      <Events>
        <Event EffectTime="OnStartTurn">
          <Conditions>
            <IsFalse Value="IsEnemyTurn"/>
            <IsTrue Value="CurrentNightHour = NightThreshold"/>
          </Conditions>
          <Actions>
            <InstantiateStatEffectDisplay Stat="ActionPoints" Value="ActionPointsBonus"/>
            <InstantiateStatEffectDisplay Stat="MovePoints" Value="MovePointsBonus"/>
          </Actions>
        </Event>
      </Events>
      <Conditions>
        <IsTrue Value="TriggerCondition"/>
      </Conditions>
      <Effects>
        <StatModifier Stat="ActionPointsTotal" Value="ActionPointsBonus"/>
        <StatModifier Stat="MovePointsTotal" Value="MovePointsBonus"/>
      </Effects>
    </BufferModule>
  </Modules>
</PerkDefinition>

```

And voilà, the NightOwl perk is finished!

Level: Medium – Duelist



The perk Duelist (*One By One* in game) is not a particularly hard perk to integrate, but it has a lot of small things and therefore needs a lot of events.

First Step: Id and variables

I like to start with defining my variables. As usual, the more the variables the better. A perk having every information cut down into small variables is easier to maintain and is easy to handle.

Here, the basic variables are **BonusMax**, **PhysicalMultiplierValue** and **DefaultMultiplierValue**. They are the heart of the perk and will be used for sure in the Localization to describe the perk.

Once the basics are defined, we are going to define two variables:

- **DynamicMultiplierValue** which returns **PhysicalMultiplierValue** if the skill deals physical damages, and **DefaultMultiplierValue** if that is not the case (see *Interpreted Methods for further information on BooleanChoice*).
- **OverAllDamageBonus** which returns the clamped total bonus offered by this perk, `Module.Buffer` being where we will store our current value.

```
<PerkDefinition Id="Duelist">
  <TokenVariables>
    <TokenVariable Key="OverallDamageBonus" Value="Min(BonusMax, Module.Buffer)"/>
    <TokenVariable Key="DynamicMultiplierValue" Value="BooleanChoice(Skill.IsPhysicalDamage,
PhysicalMultiplierValue, DefaultMultiplierValue)"/>
    <TokenVariable Key="DefaultMultiplierValue" Value="4"/>
    <TokenVariable Key="PhysicalMultiplierValue" Value="6"/>
    <TokenVariable Key="BonusMax" Value="60"/>
  </TokenVariables>
</PerkDefinition>
```

Second Step: Modules

Effects

We need our perk to give a permanent stat bonus, therefore we will be using a `StatModifier`. Since it is a “permanent effect” we do not need to trigger it, we just have to update its value. To do so, we simply use our pre-defined variable `OverallDamageBonus`, which contains our `Module.Buffer` value;

Every time the Module.Buffer value will be update, our StatModifier will be updated too automatically. By design, we usually want to update those value between two hits of a skill.

```
<PerkDefinition Id="Duelist">
  <TokenVariables>
    <TokenVariable Key="OverallDamageBonus" Value="Min(BonusMax, Module.Buffer)"/>
    <TokenVariable Key="DynamicMultiplierValue" Value="BooleanChoice(Skill.IsPhysicalDamage,
PhysicalMultiplierValue, DefaultMultiplierValue)"/>
    <TokenVariable Key="DefaultMultiplierValue" Value="4"/>
    <TokenVariable Key="PhysicalMultiplierValue" Value="6"/>
    <TokenVariable Key="BonusMax" Value="60"/>
  </TokenVariables>
  <Modules>
    <BufferModule>
      <Effects>
        <StatModifier Stat="OverallDamage" Value="OverallDamageBonus"/>
      </Effects>
    </BufferModule>
  </Modules>
</PerkDefinition>
```

Events

Since we do not want to update the perk on each kill/hit but only between two multi-hit of a skill, we will store the hit/kill counts and use the OnSkillNextHit event to update the perk using those stored values. Because of this, the perk will need a good amount of events, so we will deal with them one by one here.

Also, you will see for the first time the use of the Module's buffers. We will use all three of them:

- Module.Buffer will be used to store the current overall damage bonus. This value will only change between each multi hit of a skill, not during the hits, as we do not want the bonus to be changing from target to target.
- Module.Buffer2 will be used to store the bonus overall damaged that our current hit generated.
- Module.Buffer3 will be used to know if the current hit should trigger a reset of the perk or not.

We could have used only two buffers, merging the second and the third together, but for clarity reasons we decided to stick with one role per buffer here.

Event – Morning reset

The easiest one is the regular morning reset:

```
<Event EffectTime="OnStartProductionTurn">
  <Actions>
    <SetBufferTo Value="0"/>
  </Actions>
</Event>
```

Every time the production turn will start, we will reset the perk's Module.Buffer value to 0.

Event – Remember each isolated target kill

The main behavior of the perk is: when killing an isolated unit, gain +4% or +6% OverallDamage (up to 60%). To do so, we will use the OnTargetKill event, check if the target is valid, and then update our values.

Before anything, we need to see if a few conditions are true:

- First of all, we make sure we have some event data to work with by using the HasPerkData security (*in this example, this will always be true since the OnTargetKilled event is always triggered by a skill, which automatically creates data for us*).
- Second, we make sure the data contains a skill by using the HasSkill security (*in this example, this will always be true since the OnTargetKilled event is always triggered by a skill, which automatically creates data for us*).
- Third, we first make sure that we killed a unit by using the IsTargetUnit security.
- Now that we know we killed a unit, we make sure it is isolated.
- Finally, we make sure that our current bonus is not already at the maximum.

Once all those conditions are verified, we increase our Module.Buffer2 by DynamicMultiplierValue, which automatically returns +4 or +6 depending on the skill damage type.

```
<Event EffectTime="OnTargetKilled">
  <Conditions>
    <IsTrue Value="HasPerkData"/>
    <IsTrue Value="HasSkill"/>
    <IsTrue Value="IsTargetUnit"/>
    <IsTrue Value="TargetUnit.IsIsolated"/>
    <IsTrue Value="BonusMax > OverallDamageBonus"/>
  </Conditions>
  <Actions>
    <IncreaseBuffer BufferIndex="Buffer2" Value="DynamicMultiplierValue"/>
  </Actions>
</Event>
```

Event – Remember if a non-isolated target has been hit

Now that we handled the case for killing an isolated unit, we have to handle the case where we hit an unit that isn't isolated. To do so, we will use the OnTargetHit event and look at the target we just hit.

We need to see if a few conditions are true:

- First of all, we make sure we have some event data to work with by using the HasPerkData security *(in this example, this will always be true since the OnTargetKilled event is always triggered by a skill, which automatically creates data for us).*
- Second, we make sure the data contains a skill by using the HasSkill security *(in this example, this will always be true since the OnTargetKilled event is always triggered by a skill, which automatically creates data for us).*
- Finally, we verify that the unit we hit is **not** isolated.

If all those conditions are verified, we know we just hit a non-isolated enemy. So, we will store in the Module.Buffer3 the value -1.

```
<Event EffectTime="OnTargetHit">
  <Conditions>
    <IsTrue Value="HasPerkData"/>
    <IsTrue Value="IsTargetUnit"/>
    <IsFalse Value="TargetUnit.IsIsolated"/>
  </Conditions>
  <Actions>
    <SetBufferTo BufferIndex="Buffer3" Value="-1"/>
  </Actions>
</Event>
```

Event – Increase the current bonus if we need to

As specified above, we want to update our perk between two multi hits. In order to do so, we will be using the OnSkillNextHit event.

We will start with the case where no reset is needed. Each event can only have one purpose, so we will need to create another OnSkillNextHit event to handle the reset.

So, in the event where Module.Buffer3 equals 0 (meaning the skill is valid and does not triggers the perk reset) and where Module.Buffer2 is superior to 0 (meaning we killed at least one isolated unit and we are not already at the maximum bonus), we want to do a few things:

- First, we want to instantiate a visual feedback to give the information to the player that the perk values just got updated.
- Then, we want to increase Module.Buffer by Module.Buffer2. This will automatically update the StatModifier values, since the effect is using Module.Buffer as well.
- Finally, we want to reset Module.Buffer2 to 0, since we will be using it again for the next hit or the next skill.

```
<Event EffectTime="OnSkillNextHit">
  <Conditions>
    <IsTrue Value="Module.Buffer3 = 0"/> <!-- Attack is valid -->
    <IsTrue Value="Module.Buffer2 > 0"/>
  </Conditions>
  <Actions>
    <InstantiateStatEffectDisplay Stat="OverallDamage" Value="Min(BonusMax - OverallDamageBonus,
Module.Buffer2)"/> <!-- In case Buffer2 overshoots BonusMax -->
    <IncreaseBuffer BufferIndex="Buffer" Value="Module.Buffer2"/>
    <SetBufferTo BufferIndex="Buffer2" Value="0"/>
  </Actions>
</Event>
```

Event – Reset the perk if we need to

Now, In the event where Module.Buffer3 (the buffer where we stored our non-isolated hit) equals -1, we need to reset the perk buffers.

Before resetting all the buffers, we want to give the information to the player that he just lost the bonus the perk used to give him. To do so, we are instantiating a effect display with the current bonus multiplied by -1 to show the loss.

Then we reset the perk by simply setting all the buffers to 0.

```
<Event EffectTime="OnSkillNextHit">
  <Conditions>
    <IsTrue Value="Module.Buffer3 = -1"/> <!-- Attack isn't valid -->
  </Conditions>
  <Actions>
    <InstantiateStatEffectDisplay Stat="OverallDamage" Value="OverallDamageBonus * -1"/>
    <SetBufferTo BufferIndex="Buffer" Value="0"/>
    <SetBufferTo BufferIndex="Buffer2" Value="0"/>
    <SetBufferTo BufferIndex="Buffer3" Value="0"/>
  </Actions>
</Event>
```


Events – Final result

With all those events, we covered all possible outcomes and the entire behavior of the perk is done.

```
<Events>
  <Event EffectTime="OnTargetKilled">
    <Conditions>
      <IsTrue Value="HasPerkData"/>
      <IsTrue Value="HasSkill"/>
      <IsTrue Value="IsTargetUnit"/>
      <IsTrue Value="TargetUnit.IsIsolated"/>
      <IsTrue Value="BonusMax > OverallDamageBonus"/>
    </Conditions>
    <Actions>
      <IncreaseBuffer BufferIndex="Buffer2" Value="DynamicMultiplierValue"/>
    </Actions>
  </Event>
  <Event EffectTime="OnTargetHit">
    <Conditions>
      <IsTrue Value="HasPerkData"/>
      <IsTrue Value="IsTargetUnit"/>
      <IsFalse Value="TargetUnit.IsIsolated"/>
    </Conditions>
    <Actions>
      <SetBufferTo BufferIndex="Buffer3" Value="-1"/>
    </Actions>
  </Event>
  <Event EffectTime="OnSkillNextHit">
    <Conditions>
      <IsTrue Value="Module.Buffer3 = 0"/> <!-- Attack is valid -->
      <IsTrue Value="Module.Buffer2 > 0"/>
    </Conditions>
    <Actions>
      <InstantiateStatEffectDisplay Stat="OverallDamage" Value="Min(BonusMax - OverallDamageBonus,
Module.Buffer2)"/> <!-- In case Buffer2 overshoots BonusMax -->
      <IncreaseBuffer BufferIndex="Buffer" Value="Module.Buffer2"/>
      <SetBufferTo BufferIndex="Buffer2" Value="0"/>
    </Actions>
  </Event>
  <Event EffectTime="OnSkillNextHit">
    <Conditions>
      <IsTrue Value="Module.Buffer3 = -1"/> <!-- Attack isn't valid -->
    </Conditions>
    <Actions>
      <InstantiateStatEffectDisplay Stat="OverallDamage" Value="OverallDamageBonus * -1"/>
      <SetBufferTo BufferIndex="Buffer" Value="0"/>
      <SetBufferTo BufferIndex="Buffer2" Value="0"/>
      <SetBufferTo BufferIndex="Buffer3" Value="0"/>
    </Actions>
  </Event>
  <Event EffectTime="OnStartProductionTurn">
    <Actions>
      <SetBufferTo Value="0"/>
    </Actions>
  </Event>
</Events>
```

Third Step: View

Now that we have a working perk, we want to give as much information as possible in the view component.

- Bonus will contain the current bonus given by the perk.
- Highlight will contain the “good” conditions regarding the perk (the ones that does not reset your bonus).
- Since we are dealing with Isolated enemies, we want to explain this concept thanks to a CompendiumEntry.

```
<PerkDefinition Id="Duelist">
  <TokenVariables>
    <TokenVariable Key="OverallDamageBonus" Value="Min(BonusMax, Module.Buffer)"/>
    <TokenVariable Key="DynamicMultiplierValue" Value="BooleanChoice(Skill.IsPhysicalDamage,
PhysicalMultiplierValue, DefaultMultiplierValue)"/>
    <TokenVariable Key="DefaultMultiplierValue" Value="4"/>
    <TokenVariable Key="PhysicalMultiplierValue" Value="6"/>
    <TokenVariable Key="BonusMax" Value="60"/>
  </TokenVariables>
  <View>
    <DisplayInHUD>
      <Bonus Value="OverallDamageBonus"/>
    </DisplayInHUD>
    <Highlight>
      <IsTrue Value="Skill.IsAttack"/>
      <IsTrue Value="IsTargetUnit"/>
      <IsTrue Value="TargetUnit.IsIsolated"/>
    </Highlight>
    <CompendiumEntries>
      <CompendiumEntry Id="GameConcept_Isolated"/>
    </CompendiumEntries>
  </View>
  <Modules>
    <...>
  </Modules>
</PerkDefinition>
```

Fourth Step: Localization

The final step is the Localization step. Since we were intelligent enough defining as much things as possible in separated variables, this will be very straight forward.

Here is the Localization keys and values for the Duelist description:

KEY	VALUE
PerkDescription_Duelist	Whenever an isolated unit is killed : {0}, or {1} when using {2}, up to {3}. Hitting a non isolated unit resets the bonus. This modifier is reset every morning.
PerkEffectInformations_Duelist	{4}

You can see that we are using a detailed description through the “PerkEffectInformations_Duelist” localization value. This part of the description will only be displayed after the perk purchase.

Since our localization adds words between the values and the affected stats, we need to use a combination of LocArgument and StatArgument.

- LocArguments will be used with a prefix, a suffix and/or a specific style.
- Some StatArgument will be used without a value, in order to only print the stylized stat name. Some will be used in the usual way.

Here are our 4 localization arguments, integrated in order:

```

<PerkDefinition Id="Duelist">
  <TokenVariables>
    <TokenVariable Key="OverallDamageBonus" Value="Min(BonusMax, Module.Buffer)"/>
    <TokenVariable Key="DynamicMultiplierValue" Value="BooleanChoice(Skill.IsPhysicalDamage,
PhysicalMultiplierValue, DefaultMultiplierValue)"/>
    <TokenVariable Key="DefaultMultiplierValue" Value="4"/>
    <TokenVariable Key="PhysicalMultiplierValue" Value="6"/>
    <TokenVariable Key="BonusMax" Value="60"/>
  </TokenVariables>
  <LocArguments>
    <StatArgument Stat="OverallDamage" Value="DefaultMultiplierValue"/>
    <LocArgument Value="PhysicalMultiplierValue" Prefix="+" Suffix="%" Style="GoodNb"/>
    <StatArgument Stat="PhysicalDamage"/>
    <LocArgument Value="BonusMax" Suffix="%" Style="KeyWordNb"/>
    <StatArgument Stat="OverallDamage" Value="OverallDamageBonus" Interpreted="true"/>
  </LocArguments>
  <View>
    <DisplayInHUD>
      <Bonus Value="OverallDamageBonus"/>
      <Highlight>
        <IsTrue Value="Skill.IsAttack"/>
        <IsTrue Value="IsTargetUnit"/>
        <IsTrue Value="TargetUnit.IsIsolated"/>
      </Highlight>
    </DisplayInHUD>
    <CompendiumEntries>
      <CompendiumEntry Id="GameConcept_Isolated"/>
    </CompendiumEntries>
  </View>
  <Modules>
    <...>
  </Modules>
</PerkDefinition>

```

And with that, the perk is done!

Conclusion

If you have any question regarding Perks, feel free to come on Discord and chat with the other modders. If nobody is able to answer you, the developers are here to help, but it will depend on their availability. Depending on the complexity of your inquiries, designers might also be able to give you an answer.